

Modelado de reglas de negocio

Una estructura común para su extracción automatizada
desde código heredado mediante grafos de conocimiento

Diego Mosquera

PwC, AI Factory

8 de junio de 2026

Resumen

La modernización de sistemas de core bancario heredados constituye uno de los retos contemporáneos más complejos de la ingeniería de software, en parte porque las reglas de negocio que rigen las operaciones críticas se encuentran embebidas en el código fuente legacy, principalmente en COBOL sobre frameworks como Altamira. Extraer y formalizar estas reglas es indispensable para procesos de modernización, auditoría regulatoria y transformación digital. Este artículo plantea y prueba la hipótesis de que las reglas de negocio bancarias, pese a su diversidad funcional, comparten una estructura conceptual común que puede ser caracterizada por un núcleo definido de siete dimensiones de información. Para sustentar esta hipótesis se examinan los estándares OMG SBVR y DMN, así como el Business Rules Manifesto de Ronald Ross, y se demuestra que sus elementos fundamentales convergen en un conjunto común y operacionalizable. A partir de esta convergencia, se propone un metamodelo semántico que permite representar el código de sistemas de core bancario como un grafo de conocimiento en Neo4j, integrando las cuatro capas características del framework Altamira: código fuente, parametría, repositorio de componentes y orquestación batch. Sobre este grafo, la extracción automatizada de reglas se realiza en dos tiempos: primero, una consulta de detección identifica candidatos mediante un patrón estructural; segundo, siete consultas adicionales recuperan las dimensiones del núcleo común para cada candidato, produciendo paquetes contextuales auto-contenidos. Se ilustra el enfoque con un caso completo de transferencia bancaria entre cuentas propias del titular, demostrando la viabilidad práctica del método y su independencia respecto a las particularidades de un sistema concreto.

Palabras clave: reglas de negocio, sistemas heredados, COBOL, Altamira, grafo de conocimiento, Neo4j, ingeniería inversa, SBVR, DMN, metamodelo semántico, modernización bancaria.

1. Introducción

Los sistemas de core bancario constituyen la columna vertebral operativa de la mayoría de las instituciones financieras del mundo. Una proporción significativa de estos sistemas fue desarrollada hace décadas sobre tecnologías como COBOL, CICS y DB2, ejecutándose sobre infraestructura mainframe [6]. Aunque maduros y robustos, estos sistemas presentan desafíos serios para los procesos contemporáneos de modernización: el conocimiento de negocio se encuentra mayoritariamente *embebido* en el código fuente, sin documentación funcional actualizada que lo represente de forma independiente.

En el caso específico de los frameworks bancarios especializados, como Altamira, esta dificultad se amplifica. Las reglas que gobiernan operaciones críticas (validaciones, autorizaciones, cálculos de comisiones, controles de límites) no residen únicamente en el código COBOL: dependen también de tablas paramétricas almacenadas en DB2, del repositorio de componentes que resuelve la indirección entre operativas lógicas y programas físicos, y de cadenas batch orquestadas por planificadores como Control-M u OPC. Esta naturaleza multicapa convierte la extracción de reglas de negocio en un problema de ingeniería inversa cuyo abordaje tradicional, basado en lectura manual del código por parte de analistas funcionales, resulta inviable a escala.

La importancia de disponer de una representación formal y consultable de las reglas de negocio bancarias trasciende el caso de uso puntual de modernización. La regulación financiera contemporánea, en particular los principios del Comité de Basilea para la agregación efectiva de datos de riesgo (BCBS 239) [7], exige a las entidades trazabilidad y consistencia en los criterios aplicados a las operaciones críticas. La capacidad de responder con precisión a preguntas como «¿qué reglas se aplican a esta operación y bajo qué condiciones?» se ha vuelto un requerimiento regulatorio explícito.

Frente a este escenario, la literatura y la práctica industrial han desarrollado diversos enfoques. Por un lado, los estándares de la Object Management Group (OMG) —SBVR [1] y DMN [2]— proporcionan vocabularios formales para representar reglas y decisiones de negocio. Por otro, el campo del análisis estático de programas ha producido representaciones intermedias unificadas como el Code Property Graph [5], capaces de soportar análisis profundo sobre grandes volúmenes de código. Sin embargo, ambos cuerpos de conocimiento han evolucionado de forma relativamente independiente: los estándares de reglas describen *qué* es una regla pero no *cómo* extraerla de código heredado; las técnicas de análisis estático describen *cómo* analizar código pero no *qué* dimensiones funcionales caracterizan una regla bancaria.

El presente trabajo se sitúa en la convergencia entre ambos campos. Propone que las reglas de negocio bancarias, lejos de ser un universo heterogéneo e irreducible, presentan una estructura conceptual común caracterizable a partir de la combinación de los estándares mencionados. Sobre esa caracterización, se construye un metamodelo semántico que habilita la representación del código de un sistema de core bancario como un grafo de conocimiento, y se diseña un procedimiento de extracción de reglas en dos tiempos. La contribución se valida sobre un caso completo derivado de la operativa real de transferencia entre cuentas propias de un cliente bancario.

2. Planteamiento del problema e hipótesis

2.1 Contexto del problema

El problema concreto abordado puede formularse de la siguiente manera. Dado un sistema de core bancario implementado sobre el framework Altamira —o uno equivalente con características arquitectónicas similares—, se requiere extraer, formalizar y catalogar las reglas de negocio que rigen sus operaciones. Esta extracción debe ser:

1. **Automatizada**, dada la escala del problema (típicamente miles de programas COBOL en un banco grande).
2. **Trazable**, manteniendo el vínculo entre cada regla extraída y su evidencia en código fuente, parametría y configuración.
3. **Funcional**, expresando las reglas en un lenguaje accesible a analistas de negocio, no en terminología técnica COBOL.
4. **Auditable**, sosteniendo trazabilidad regulatoria sobre las decisiones automatizadas que aplica el sistema.

El sistema Altamira presenta, además, una particularidad estructural que condiciona el problema. La realidad operativa del sistema no se agota en el código COBOL, sino que se distribuye en cuatro capas interdependientes: **(C1)** código fuente COBOL, JCL y copybooks; **(C2)** tablas paramétricas en DB2 que contienen los valores aplicables; **(C3)** el repositorio Altamira que establece la jerarquía operativa-programa físico; y **(C4)** el planificador batch que orquesta la ejecución temporal de procesos. Una regla de negocio típica involucra elementos de varias de estas capas simultáneamente.

2.2 Hipótesis

Sobre la base del problema descrito, este trabajo plantea la siguiente hipótesis principal:

H1. *Las reglas de negocio bancarias, pese a su diversidad funcional, comparten una estructura conceptual común caracterizable por un núcleo finito de dimensiones de información, las cuales son derivables a partir de la convergencia de los estándares formales contemporáneos de modelado de reglas (SBVR, DMN) y el Business Rules Manifesto.*

A partir de H1, se derivan dos hipótesis operativas que son verificables empíricamente:

H2. *El núcleo común de elementos identificado bajo H1 puede ser instrumentado como un metamodelo semántico que permita representar el código y los catálogos de un sistema de core bancario como un grafo de conocimiento.*

H3. *Sobre el grafo de conocimiento resultante, la extracción automatizada de reglas de negocio puede realizarse en dos tiempos: una consulta única que identifica candidatos mediante un patrón estructural, y siete consultas adicionales que mapean cada candidato al núcleo común de dimensiones.*

2.3 Alcance

El alcance del presente artículo se limita a la formulación teórica del marco, la propuesta del metamodelo semántico y las consultas asociadas, y la validación de las hipótesis mediante un caso completo. Quedan explícitamente fuera de alcance: la implementación detallada de un ingestor que pueble el grafo a partir de código fuente real, la integración con modelos de lenguaje para la redacción final de las reglas, y la evaluación cuantitativa sobre un repositorio bancario completo, los cuales constituyen líneas de trabajo futuro.

3. Marco referencial

El marco conceptual de este trabajo se apoya en cuatro cuerpos de conocimiento formal cuya combinación es necesaria para sostener la hipótesis principal: los estándares OMG para modelado de reglas (SBVR y DMN), el corpus doctrinario de Ronald Ross sobre el enfoque de reglas de negocio, las técnicas de representación de programas mediante Code Property Graphs, y el marco regulatorio bancario contemporáneo.

3.1 SBVR: vocabulario semántico de reglas de negocio

El estándar SBVR [1], cuyas bases conceptuales se desarrollaron previamente en los trabajos del Business Rules Group [8], establece una taxonomía formal para representar reglas y vocabulario de negocio independientemente de su implementación tecnológica. Sus elementos fundamentales son:

- **Business vocabulary**: un conjunto de términos (*noun concepts, verb concepts*) que constituyen el vocabulario del dominio.
- **Fact types**: relaciones entre términos del vocabulario que expresan hechos del negocio.
- **Operative rules**: reglas que prescriben o prohíben acciones, y cuya violación es procesable por el sistema.
- **Structural rules**: reglas que definen lo que es necesariamente verdadero en el dominio.

SBVR introduce además la noción de *community of practice* y *authoritative source*, que serán relevantes para sustentar la trazabilidad de las reglas.

3.2 DMN: modelo y notación de decisiones

El estándar DMN [2] provee una notación formal para representar decisiones automatizables. A diferencia de SBVR, que privilegia el vocabulario, DMN se centra en la lógica decisional. Sus elementos fundamentales son:

- **Decision**: una unidad de toma de decisión que produce un resultado a partir de entradas.
- **Input data**: los datos requeridos como entrada para una decisión.
- **Knowledge source**: la fuente autoritativa de la lógica decisional (regulación, política corporativa, etc.).
- **Business Knowledge Model (BKM)**: lógica reutilizable que puede ser invocada desde una decisión.
- **Decision Requirements Diagram (DRD)**: representación gráfica de las dependencias entre decisiones.
- **Decision table**: notación tabular para expresar la lógica concreta.

3.3 Business Rules Manifesto y enfoque de Ross

El *Business Rules Manifesto* [3] establece diez principios sobre la naturaleza y el tratamiento de las reglas de negocio. Entre ellos, los más relevantes para el presente trabajo son: **(M1)** las reglas son ciudadanos de primera clase del modelo de información; **(M2)** las reglas deben expresarse declarativamente en lenguaje natural estructurado; **(M3)** las reglas son independientes de las aplicaciones que las implementan; **(M4)** las reglas deben ser explícitas. La obra de Ross [4] desarrolla estos principios en un enfoque operativo, donde una regla se caracteriza siempre por un *antecedente* (condición), un *consecuente* (acción o restricción) y una *justificación* (fuente autoritativa). Trabajos contemporáneos como el de Date [10] reforzaron este enfoque desde la perspectiva del modelado relacional, contribuyendo a consolidar la noción de regla de negocio como entidad de primera clase en el diseño de sistemas.

3.4 Code Property Graph

Yamaguchi et al. [5] introdujeron el Code Property Graph (CPG) como una representación unificada que combina, en un único grafo, el árbol sintáctico abstracto (AST), el grafo de flujo de control (CFG) y el grafo de dependencias de programa (PDG). Esta unificación habilita análisis que requieren cruzar dimensiones sintácticas, de control y de datos simultáneamente. Si bien el CPG fue originalmente propuesto para análisis de vulnerabilidades de seguridad, su modelo es directamente aplicable a la extracción de reglas de negocio. Trabajos posteriores en metamodelos de descubrimiento de conocimiento sobre sistemas heredados [11] y herramientas específicas para COBOL [13] confirman la viabilidad de este enfoque sobre lenguajes de generación previa.

3.5 Marco regulatorio bancario

Finalmente, el documento BCBS 239 del Comité de Basilea [7] establece los principios para la agregación efectiva de datos de riesgo y reporte. Sus principios de *precisión e integridad*, *completitud* y *trazabilidad* convierten a la extracción formal de reglas de negocio en un requerimiento regulatorio, no meramente técnico. Esto refuerza la necesidad de que cualquier metodología propuesta mantenga vínculo explícito entre las reglas extraídas y sus fuentes de evidencia.

4. Estructura de las reglas de negocio

Esta sección demuestra cómo los elementos provenientes de SBVR, DMN y el enfoque de Ross convergen en un núcleo común de dimensiones, sosteniendo así la hipótesis principal H1.

4.1 Análisis comparativo de los elementos fundamentales

Una regla de negocio, según los marcos referenciales examinados, queda completamente caracterizada cuando se especifican los siguientes aspectos: (a) *dónde* reside, (b) *qué* la implementa concretamente, (c) *qué información* consulta para decidir, (d) *bajo qué condiciones* se aplica, (e) *qué efectos* produce, (f) *cómo se relaciona* con otras reglas o procesos, y (g) *en qué lenguaje funcional* se expresa. Estas siete dimensiones, que en lo sucesivo denominaremos *el núcleo común*, pueden mapearse a elementos formales de los estándares referenciados como se muestra en la Tabla 1.

Cuadro 1: Mapeo del núcleo común con los elementos de SBVR, DMN y el enfoque de Ross.

Dimensión del núcleo	SBVR	DMN	Ross
D1. Alcance (<i>scope</i>)	Community of practice	Knowledge source	Contexto autoritativo
D2. Implementación (<i>code slice</i>)	Concrete representation	BKM body	Realización técnica
D3. Datos de entrada (<i>data context</i>)	Vocabulary, fact types	Input data	Antecedente (datos)
D4. Condición decisional (<i>decision</i>)	Operative rule	Decision table	Antecedente (lógica)
D5. Efectos producidos (<i>effects</i>)	Action (necessity / possibility)	Decision output	Consecuente
D6. Contexto temporal/batch	— (extensión)	DRD (extendido)	Workflow contextual
D7. Vocabulario funcional (<i>glossary</i>)	Business vocabulary	Glossary	Lenguaje declarativo

La Tabla 1 muestra que las siete dimensiones tienen correspondencia explícita en al menos dos de los tres marcos formales examinados. La única dimensión sin equivalente directo en SBVR es D6 (contexto batch), que en este trabajo se incorpora como extensión específica del dominio bancario, sustentada en el DRD de DMN ampliado para considerar dependencias batch temporales.

4.2 Verificación de la hipótesis sobre la estructura común

La existencia de la correspondencia mostrada en la Tabla 1 permite enunciar formalmente el resultado.

Proposición. *Toda regla de negocio bancaria R puede caracterizarse completamente mediante una tupla $\langle D_1, D_2, D_3, D_4, D_5, D_6, D_7 \rangle$ donde cada D_i corresponde*

a una de las siete dimensiones del núcleo común. Las dimensiones D_1 a D_5 y D_7 son obligatorias; D_6 es opcional y aplica únicamente cuando la regla tiene continuación en procesos batch.

Esta caracterización sostiene la hipótesis H1: las reglas bancarias comparten una estructura común finita y bien definida. La estructura no es arbitraria sino derivada de marcos formales reconocidos por la industria. Las dimensiones son ortogonales (cada una aporta información que las otras no proveen) y exhaustivas en el sentido de que cubren los requerimientos planteados por el marco regulatorio BCBS 239 [7] sobre trazabilidad y completitud.

4.3 Operacionalización del núcleo común

La caracterización anterior es de utilidad teórica, pero requiere operacionalización para hacerse manipulable computacionalmente. En las secciones siguientes se mostrará que cada dimensión D_i admite una formulación como consulta declarativa sobre un grafo de conocimiento adecuadamente construido. Esto enlaza directamente con la hipótesis H2: el núcleo común habilita la definición de un metamodelo semántico, y con H3: las siete dimensiones se mapean uno a uno con siete consultas de extracción.

5. Grafos de conocimiento para representar reglas de negocio

Un *grafo de conocimiento* es una representación estructurada en la que entidades del dominio se modelan como nodos, las relaciones entre ellas como aristas tipadas, y los atributos como propiedades sobre nodos y aristas. Esta representación resulta particularmente adecuada para el problema de extracción de reglas de negocio bancarias por tres razones convergentes.

Razón 1: la naturaleza intrínsecamente relacional del dominio. Las reglas no existen como entidades aisladas: dependen del programa que las contiene, del párrafo que las implementa, de las variables que consultan, de las tablas que leen o escriben, de los términos funcionales que las describen. Una representación tabular relacional clásica fragmentaría estas relaciones en múltiples tablas y joins. Una representación en grafo expresa naturalmente esta interconexión.

Razón 2: la equivalencia con el Code Property Graph. Como se mencionó en el marco referencial, Yamaguchi et al. [5] demostraron que la unificación de AST, CFG y PDG en un único grafo habilita análisis que serían intratables con representaciones aisladas. El metamodelo propuesto en este trabajo extiende esta tradición al combinar las capas técnicas (código) con las capas funcionales (parametría, repositorio, batch, glosario) características del entorno Altamira.

Razón 3: la expresividad de los lenguajes de consulta sobre grafos. Los lenguajes contemporáneos de consulta sobre grafos, en particular Cypher [12], permiten expresar de manera declarativa patrones estructurales complejos que en SQL requerirían joins anidados extensos. Patrones como «*todos los párrafos que contienen una decisión que produce un código de retorno*» son expresables en Cypher en una sola línea legible.

Para los propósitos de este trabajo se selecciona **Neo4j** como plataforma de implementación del grafo de conocimiento, atendiendo a su madurez, su lenguaje Cypher como estándar industrial de facto, y su disponibilidad en versión comunitaria. Las consideraciones presentadas en las secciones siguientes son independientes de la elección concreta de la plataforma y aplicables a cualquier base de grafos compatible.

6. Metamodelo semántico

Esta sección desarrolla en detalle la pieza central del enfoque propuesto: el metamodelo semántico que permite representar un sistema de core bancario como un grafo de conocimiento sus-

ceptible de ser consultado para extraer reglas. Se aborda su definición, su justificación, sus principios de diseño, y su aplicación concreta sobre el grafo.

6.1 Definición

Se denomina *metamodelo semántico* a la especificación formal de los tipos de entidades, los tipos de relaciones y las propiedades obligatorias y opcionales que estructuran el grafo de conocimiento, junto con las invariantes que el grafo debe satisfacer. El metamodelo no contiene datos concretos: contiene la *forma* que esos datos deben tomar para que las consultas posteriores puedan operar sobre ellos.

A diferencia de un esquema relacional clásico, que se materializa íntegramente como restricción de la base de datos, el metamodelo semántico se materializa parcialmente. Una porción —constraints de unicidad, índices— es enforced directamente por el motor Neo4j. Otra porción —propiedades obligatorias, vocabulario de etiquetas semánticas, cardinalidad de relaciones— vive como convención documentada que el componente de ingesta debe respetar y que los componentes de consulta asumen como verdadera.

6.2 Por qué hace falta un metamodelo

La necesidad del metamodelo deriva directamente de la hipótesis H1. Si las reglas bancarias comparten una estructura común caracterizable por siete dimensiones, entonces el grafo que las represente debe contener, de forma sistemática y consistente, la información necesaria para responder esas siete dimensiones para cualquier regla. Sin un metamodelo, cada ingesta de código produciría un grafo con estructura distinta, y las consultas no podrían operar uniformemente sobre todos los sistemas.

Además, el dominio bancario plantea exigencias específicas que un metamodelo genérico de análisis de programas no cubre: la jerarquía multi-país, la indirección operativa-programa, los snapshots paramétricos con vigencia temporal, las cadenas batch con dependencias técnicas y lógicas. El metamodelo semántico debe incorporar estas particularidades sin perder la base formal heredada del análisis estático.

6.3 Principios de diseño

El metamodelo semántico propuesto se rige por seis principios:

1. **Minimalidad suficiente.** Cada nodo y cada arista deben justificarse respondiendo a la pregunta «¿qué *dimensión del núcleo común habilitan?*». Elementos sin justificación no se incorporan.
2. **Separación entre estructura y semántica.** Las variables COBOL se representan como entidades técnicas (`DataItem`); los conceptos de negocio se representan como entidades funcionales (`DomainTerm`). La relación entre ambas es explícita, no implícita.
3. **Etiquetado semántico controlado.** Para distinguir el rol funcional de las variables (código de retorno, importe, estado, etc.) se utiliza una propiedad `semantic_tag` con un vocabulario controlado de unos quince valores. Esto desacopla la lógica de las consultas de la nomenclatura particular de cada banco.
4. **Identificación estable.** Cada nodo cuenta con un identificador determinístico y trazable a la fuente, de la forma `PROGRAMA::ELEMENTO`.
5. **Multi-tenancy.** La jerarquía de país es un nodo de primera clase desde la raíz, anticipando despliegues multi-jurisdiccionales.

6. **Modelado temporal explícito.** Los snapshots paramétricos llevan la fecha de extracción como propiedad, registrando explícitamente el momento al que corresponde el estado del grafo.

6.4 Nodos del metamodelo

Los nodos del metamodelo se organizan en cuatro grupos correspondientes a las cuatro capas Altamira. La Tabla 2 los presenta junto con las dimensiones del núcleo común que cada uno habilita.

Cuadro 2: Nodos del metamodelo y dimensiones del núcleo común que habilitan.

Capa	Nodo	Habilita
C3	Country	D1 (scope)
C3	Application	D1
C3	Operation	D1
C1	Program	D1, D5
C1	Paragraph	D1, D2, D4
C1	DataItem	D3, D4, D5, D7
C1	Decision	D4, D5
C2	ParameterTable	D3
C2	ParameterEntry	D3
C1/C2	Table (DB2)	D3, D5
C4	BatchJob	D6
—	DomainTerm	D7

6.5 Aristas del metamodelo

Las aristas que conectan estos nodos se presentan en la Tabla 3, agrupadas por su función.

Cuadro 3: Aristas del metamodelo, agrupadas por función.

Función	Arista	Conecta
Jerarquía Altamira	HAS_APPLICATION	Country → Application
Jerarquía Altamira	HAS_OPERATION	Application → Operation
Indirección dinámica	EXECUTES_VIA	Operation → Program
Composición de código	CONTAINS	Program → Paragraph
Decisión lógica	HAS_DECISION	Paragraph → Decision
Producción de resultado	LEADS_TO	Decision → DataItem
Uso de datos	USES	Paragraph → DataItem
Acceso a tablas	READS	Paragraph/BatchJob → Table
Escritura a tablas	WRITES, UPDATES, INSERTS	Paragraph → Table
Parametría	HAS_ENTRY	ParameterTable → ParameterEntry
Cadena batch técnica	PRECEDED_BY	BatchJob → BatchJob
Cadena batch lógica	TRIGGERS	BatchJob → BatchJob
Maapeo funcional	HAS_DOMAIN_TERM	DataItem → DomainTerm

6.6 Cómo se usa en el grafo de conocimiento

El metamodelo se utiliza en el grafo de conocimiento de tres maneras complementarias.

Primero, durante la fase de *ingesta*, el componente que pobla el grafo a partir del código fuente y los catálogos debe respetar las definiciones del metamodelo: crear nodos con los labels indicados, conectarlos con las aristas correspondientes, asignar las propiedades obligatorias. Esta responsabilidad recae sobre el ingestor, no sobre el motor Neo4j.

Segundo, durante la fase de *consulta*, las queries Cypher expresan patrones en términos del metamodelo. Una consulta como `MATCH (par:Paragraph)-[:HAS_DECISION]-(d:Decision)` funciona sobre cualquier grafo correctamente ingestado, independientemente del banco o del sistema, porque depende del metamodelo y no de los datos concretos.

Tercero, durante la fase de *validación*, un conjunto de invariantes verifica que el grafo cargado satisface las restricciones del metamodelo. Estas invariantes pueden expresarse como consultas Cypher que deben devolver cero resultados —por ejemplo, «no debe existir un *Paragraph* sin *source_text*»—, y se ejecutan al finalizar cada ingesta.

La conjunción de las tres etapas garantiza que la hipótesis H2 se cumple en la práctica: el metamodelo no es una abstracción teórica sino un artefacto operativo que estructura el ciclo completo de vida del grafo.

7. Consultas al grafo de conocimiento

Esta sección presenta las consultas Cypher que materializan la hipótesis H3: la extracción de reglas en dos tiempos mediante una consulta de detección de candidatos y siete consultas de mapeo al núcleo común. Por razones de claridad expositiva, las consultas se presentan en su forma individual; en una implementación productiva se combinan en una consulta unificada que se discute al final de la sección.

7.1 Consulta de detección de candidatos

La detección de candidatos a regla de negocio se realiza mediante el reconocimiento de un patrón estructural específico sobre el grafo. El patrón es: un `Paragraph` que contiene una `Decision` cuyo resultado (`LEADS_TO`) es un `DataItem` cuya etiqueta semántica es `return_code`. La Consulta 1 formaliza este patrón.

```

1 MATCH (par:Paragraph)-[:HAS_DECISION]->(d:Decision)
2           -[:LEADS_TO]->(sink:DataItem)
3 WHERE sink.semantic_tag = 'return_code'
4 RETURN par.id           AS candidate_id,
5        par.name         AS paragraph_name,
6        d.id             AS decision_id,
7        d.expression      AS condition,
8        d.outcome_code    AS outcome,
9        d.rule_type       AS rule_type
10 ORDER BY par.line_start;
```

Listing 1: Q0: Detección de candidatos a regla de negocio.

Cada fila devuelta por esta consulta es un candidato cuyo identificador (`candidate_id`) se utilizará como parámetro para las siete consultas de mapeo descritas a continuación. Es importante señalar que el resultado de Q0 es *hipotético*: la detección estructural propone candidatos, pero la confirmación final como reglas de negocio genuinas requiere las dimensiones de información que aportan las consultas siguientes y, en última instancia, la validación de un experto funcional.

7.2 Las siete consultas de mapeo al núcleo común

7.2.1 Q1: alcance de la regla (dimensión D1)

Resuelve la ubicación jerárquica del candidato en el sistema Altamira: país, aplicación, operativa lógica, programa físico que la implementa, y archivo fuente.

```

1 MATCH (par:Paragraph {id: $candidate_id})
2       <-[:CONTAINS]-(prog:Program)
```

```

3      <-[:EXECUTES_VIA]-(op:Operation)
4      <-[:HAS_OPERATION]-(app:Application)
5      <-[:HAS_APPLICATION]-(country:Country)
6  RETURN country.code      AS country,
7         app.name         AS application,
8         op.name          AS operation_logical,
9         prog.name         AS program,
10        prog.version      AS program_version,
11        par.name          AS paragraph,
12        par.line_start    AS line_start,
13        par.line_end      AS line_end,
14        prog.source_file  AS source_file;

```

Listing 2: Q1: Alcance de la regla.

7.2.2 Q2: implementación en código (dimensión D2)

Recupera el código fuente del párrafo candidato, opcionalmente extendido a párrafos que influyen sobre él por dependencias de datos o control.

```

1  MATCH (sink:Paragraph {id: $candidate_id})
2  OPTIONAL MATCH (origin:Paragraph)
3      -[:DATA_DEPENDS_ON|CONTROL_DEPENDS_ON*1..4]->(sink)
4  WITH sink, collect(DISTINCT origin) AS deps
5  WITH sink, [p IN deps WHERE p IS NOT NULL] + [sink] AS slice_pars
6  UNWIND slice_pars AS par
7  RETURN DISTINCT par.name      AS paragraph_name,
8                  par.source_text AS source_text,
9                  par.line_start AS line_start,
10                 par.line_end   AS line_end
11 ORDER BY par.line_start;

```

Listing 3: Q2: Slice de código.

7.2.3 Q3: contexto de datos (dimensión D3)

Esta dimensión se descompone en dos sub-consultas: Q3a recupera las tablas paramétricas leídas junto con sus filas aplicables; Q3b recupera las tablas DB2 transaccionales (no paramétricas) leídas por el párrafo.

```

1  MATCH (par:Paragraph {id: $candidate_id})-[:READS]->(pt:ParameterTable)
2  OPTIONAL MATCH (pt)-[:HAS_ENTRY]->(pe:ParameterEntry)
3  RETURN pt.name      AS parameter_table,
4         pt.snapshot_date AS snapshot_date,
5         collect(properties(pe)) AS applicable_rows;

```

Listing 4: Q3a: Paramétrica.

```

1  MATCH (par:Paragraph {id: $candidate_id})
2  OPTIONAL MATCH (par)-[:READS]->(t:Table)
3  WHERE NOT t:ParameterTable
4  RETURN collect(DISTINCT t.name) AS tables_read;

```

Listing 5: Q3b: Tablas transaccionales.

7.2.4 Q4: condición decisional (dimensión D4)

Recupera la expresión lógica de la decisión asociada al candidato, sus operandos, el código de retorno producido y la clasificación del tipo de regla.

```

1 MATCH (par:Paragraph {id: $candidate_id})-[:HAS_DECISION]->(dec:Decision)
2 RETURN dec.id AS decision_id,
3        dec.expression AS condition,
4        dec.operands AS operands,
5        dec.rule_type AS rule_type,
6        dec.outcome_code AS outcome_code;

```

Listing 6: Q4: Decisión lógica.

7.2.5 Q5: efectos producidos (dimensión D5)

Igual que Q3, se descompone en dos: Q5a recupera el código de retorno; Q5b recupera las tablas escritas por el programa que contiene al candidato.

```

1 MATCH (par:Paragraph {id: $candidate_id})-[:HAS_DECISION]->(dec:Decision)
2 RETURN dec.outcome_code AS return_code,
3        dec.expression AS triggered_when;

```

Listing 7: Q5a: Código de retorno.

```

1 MATCH (par:Paragraph {id: $candidate_id})<-[:CONTAINS]-(prog:Program)
2 OPTIONAL MATCH (prog)-[:CONTAINS]->(:Paragraph)
3     -[:WRITES|UPDATES|INSERTS]->(t:Table)
4 RETURN collect(DISTINCT t.name) AS tables_written;

```

Listing 8: Q5b: Tablas escritas.

7.2.6 Q6: contexto batch (dimensión D6)

Recupera los jobs batch que consumen los datos producidos por el programa al que pertenece el candidato, junto con su ventana de ejecución y sus predecesores y sucesores en la cadena.

```

1 MATCH (par:Paragraph {id: $candidate_id})<-[:CONTAINS]-(prog:Program)
2 MATCH (prog)-[:CONTAINS]->(:Paragraph)
3     -[:WRITES|UPDATES|INSERTS]->(t:Table)
4     <-[:READS]-(j:BatchJob)
5 OPTIONAL MATCH (j)-[:PRECEDED_BY]->(jp:BatchJob)
6 OPTIONAL MATCH (j)-[:TRIGGERS]->(jn:BatchJob)
7 RETURN DISTINCT
8     j.name AS job_name,
9     j.schedule AS schedule,
10    j.window_start + '-' + j.window_end AS window,
11    jp.name AS preceded_by,
12    jn.name AS triggers_next;

```

Listing 9: Q6: Contexto batch.

7.2.7 Q7: glosario funcional (dimensión D7)

Recupera el mapeo de términos técnicos (variables COBOL usadas por el párrafo candidato) a sus equivalentes en lenguaje funcional de negocio.

```

1 MATCH (par:Paragraph {id: $candidate_id})-[:USES]->(di:DataItem)
2 MATCH (di)-[:HAS_DOMAIN_TERM]->(term:DomainTerm)
3 RETURN DISTINCT
4     di.name AS technical_name,
5     term.functional_name AS functional_name,
6     term.definition AS definition,
7     term.entity_type AS entity_type;

```

7.3 Consulta unificada y paquete contextual

En entornos productivos, las siete consultas se combinan en una consulta unificada que produce, en una sola ejecución, un *paquete contextual* JSON con las siete secciones correspondientes a las siete dimensiones. Este paquete tiene una estructura estable y predecible, lo que facilita su consumo por componentes downstream (por ejemplo, modelos de lenguaje que produzcan una redacción final de la regla en lenguaje natural). La sección siguiente presenta un ejemplo concreto del paquete resultante.

8. Ejemplo completo

Esta sección aplica el método propuesto a un caso real simplificado: la operativa de *transferencia entre cuentas propias del cliente* en la implementación argentina del sistema Altamira de un banco minorista.

8.1 Descripción del caso

La operativa lógica OP-TRF-PROPIA está implementada en Argentina por el programa físico ARTRFPROP01 versión 3.2. El programa contiene tres reglas de negocio embebidas, cada una en un párrafo distinto:

1. **Regla de pertenencia (VALIDAR-PERTENENCIA)**: rechaza la operación cuando las cuentas origen y destino no pertenecen al mismo cliente. Código de retorno: RECH-NO-PROPIA.
2. **Regla de estado de cuenta (VALIDAR-CUENTAS)**: rechaza si alguna de las cuentas no se encuentra en estado activo. Código: RECH-CTA-INACTIVA.
3. **Regla de límite diario (VALIDAR-LIMITE)**: rechaza si el importe solicitado supera el límite paramétrico aplicable al segmento del cliente. Código: RECH-LIMITE.

8.2 Modelado del grafo

Tras la ingesta correspondiente, el grafo de Neo4j contiene 40 nodos y 51 relaciones que representan: la jerarquía Altamira (Country AR, Application Transferencias, Operation OP-TRF-PROPIA), el programa físico y sus cinco párrafos, nueve *DataItem* con sus respectivos *semantic_tag*, una *ParameterTable* (PARAM_TRANSFER) con tres *ParameterEntry*, tres *Decision* (una por regla), tres *Table* DB2 (maestros y movimientos), tres *BatchJob* (cierre ATM, consolidación, reporte BCRA) y nueve *DomainTerm* del glosario funcional.

8.3 Ejecución de la consulta de detección

Al ejecutar la consulta Q0 (Listado 1) sobre este grafo, se obtienen exactamente tres filas, una por cada regla embebida en el programa. El resultado se presenta en la Tabla 4.

Cuadro 4: Candidatos detectados por la consulta Q0 sobre el grafo de ejemplo.

Párrafo	Condición lógica	Código de retorno
VALIDAR-CUENTAS	estado de cuenta $\neq$ ACTIVA	RECH-CTA-INACTIVA
VALIDAR-PERTENENCIA	cliente origen $\neq$ destino	RECH-NO-PROPIA
VALIDAR-LIMITE	importe $>$ límite paramétrico	RECH-LIMITE

8.4 Aplicación de las siete consultas al candidato VALIDAR-LIMITE

A continuación se ilustra el resultado consolidado de aplicar las siete consultas al candidato ARTRFPROP01::VALIDAR-LIMITE, presentado como el paquete contextual JSON resultante.

```
1 {
2   "candidate_id": "ARTRFPROP01::VALIDAR-LIMITE",
3
4   "scope": {                                     // dimensión D1
5     "country": "AR",
6     "application": "Transferencias",
7     "operation": {
8       "logical_name": "OP-TRF-PROPIA",
9       "description": "Transferencia entre cuentas propias del cliente",
10      "implemented_by": "ARTRFPROP01 v3.2"
11    },
12    "program": "ARTRFPROP01",
13    "paragraph": "VALIDAR-LIMITE",
14    "lines": "85-98",
15    "source_file": "ARTRFPROP01.cbl"
16  },
17
18  "code_slice": [{                               // dimensión D2
19    "paragraph": "VALIDAR-LIMITE",
20    "source_text": "IF WS-IMPORTE > WS-LIMITE MOVE 'RECH-LIMITE' ...",
21    "line_start": 85,
22    "line_end": 98
23  }],
24
25  "data_context": {                             // dimensión D3
26    "parameter_table": "PARAM_TRANSFER",
27    "parameter_snapshot": "2026-05-15",
28    "applicable_rows": [
29      {"tipo_oper": "TRF-PROPIA", "segmento": "RETAIL", "limite": 200000.0},
30      {"tipo_oper": "TRF-PROPIA", "segmento": "PREMIUM", "limite": 500000.0},
31      {"tipo_oper": "TRF-TERCEROS", "segmento": "RETAIL", "limite": 100000.0}
32    ],
33    "tables_read": []
34  },
35
36  "decision": {                                 // dimensión D4
37    "expression": "WS-IMPORTE > WS-LIMITE",
38    "rule_type": "amount_threshold_check",
39    "operands": ["WS-IMPORTE", "WS-LIMITE"],
40    "outcome_code": "RECH-LIMITE"
41  },
42
43  "effects": {                                  // dimensión D5
44    "return_codes": ["RECH-LIMITE"],
45    "tables_written": ["MOVIMIENTOS"]
46  },
47
48  "batch_context": {                           // dimensión D6
49    "downstream_jobs": [{
50      "name": "CONSOLIDA-TRF",
51      "schedule": "días hábiles",
52      "window": "02:00-05:00",
53      "preceded_by": "CIERRE-ATM",
54      "triggers_next": "REPORTE-REG-BCRA"
55    }]
56  },
57
58  "domain_glossary": [                         // dimensión D7
59    {"technical_name": "WS-IMPORTE", "functional_name": "importe solicitado"},
60    {"technical_name": "WS-LIMITE", "functional_name": "límite diario"},
61    {"technical_name": "WS-COD-RETORNO", "functional_name": "código de resultado"}
62  ]
63 }
```

Listing 11: Paquete contextual resultante para el candidato VALIDAR-LIMITE.

8.5 Mapeo a la regla de negocio formal

El paquete contextual obtenido permite construir, de forma directa y verificable, una redacción de la regla en lenguaje funcional. Una posible redacción, sustentada exclusivamente en las dimensiones del paquete, es:

«En el contexto de la operativa Transferencia entre cuentas propias del cliente en Argentina (implementada por el programa ARTRFPROP01 versión 3.2), cuando el cliente solicita una transferencia entre sus cuentas propias, el sistema verifica que el importe solicitado no supere el límite diario aplicable a su segmento comercial. Si el importe supera el límite (200.000 para clientes Retail, 500.000 para clientes Premium), la operación se rechaza con el código RECH-LIMITE y no se asienta movimiento en la tabla de movimientos. Si el importe respeta el límite, la transferencia se procesa y los movimientos resultantes son consolidados contablemente en el proceso batch CONSOLIDA-TRF.»

Esta redacción cubre todas las dimensiones del núcleo común. Su trazabilidad es completa: cada elemento de la redacción tiene contrapartida explícita en el paquete contextual y, a través de él, en el grafo de conocimiento. La regla así extraída satisface los requerimientos de BCBS 239 [7] sobre trazabilidad y completitud.

8.6 Verificación de la hipótesis sobre el caso

El caso completo permite verificar empíricamente las tres hipótesis:

- **H1 (estructura común)** se verifica al observar que las tres reglas embebidas en el programa, pese a tener naturalezas distintas (validación de estado, validación de identidad, validación numérica), producen paquetes contextuales con la misma estructura y completitud.
- **H2 (metamodelo)** se verifica al constatar que el grafo construido bajo el metamodelo propuesto permite responder uniformemente las siete dimensiones para los tres candidatos.
- **H3 (extracción en dos tiempos)** se verifica al obtener, mediante una única consulta de detección y siete de mapeo, paquetes contextuales completos sin necesidad de intervención manual ni lógica ad hoc.

9. Discusión

Los resultados presentados permiten posicionar el enfoque propuesto respecto a los desarrollos previos y discutir sus limitaciones.

Posicionamiento frente a herramientas comerciales. Las herramientas industriales como CAST Application Intelligence Platform, IBM Application Discovery and Delivery Intelligence (ADDI) y Micro Focus Enterprise Analyzer proveen análisis estático sobre código COBOL, pero su salida son típicamente reportes técnicos (call graphs, dependencias, métricas de complejidad) y no reglas de negocio en sentido funcional. El enfoque aquí propuesto se diferencia precisamente en este punto: la salida no son métricas técnicas sino paquetes contextuales que mapean a una estructura formal de regla, lista para alimentar procesos de modernización dirigidos por negocio.

Comparación con enfoques académicos. El metamodelo de descubrimiento de conocimiento ISO/IEC 19506 (KDM) [11] propone una representación uniforme de sistemas heredados, mientras que enfoques basados en ingeniería dirigida por modelos sobre COBOL [9] han mostrado la viabilidad de derivar abstracciones funcionales desde código heredado. El enfoque del presente trabajo es complementario a ambas líneas: mientras KDM se concentra en la

representación estructural del sistema, el metamodelo semántico propuesto incorpora explícitamente las dimensiones funcionales derivadas de SBVR y DMN, así como las particularidades del dominio bancario (paramétrica, batch, multi-país).

Limitaciones reconocidas. El enfoque presenta tres limitaciones que vale la pena explicitar. Primero, el detector de candidatos es *estructural*: identifica patrones que sugieren reglas, pero la confirmación definitiva requiere validación funcional por experto humano. Segundo, el etiquetado semántico de las variables (`semantic_tag`) es heurístico y puede generar falsos positivos o negativos en código heterogéneo. Tercero, las reglas que residen exclusivamente en datos paramétricos (sin manifestación condicional en código) no son detectadas por el patrón propuesto y requieren un detector complementario aún no formalizado.

Implicaciones para la modernización bancaria. La disponibilidad de un grafo de conocimiento consultable y un conjunto estable de consultas habilita escenarios que con metodologías tradicionales resultarían impracticables: análisis de impacto regulatorio sobre reglas extraídas, comparación funcional entre versiones de un mismo programa, migración asistida por evidencia hacia plataformas modernas, y generación de documentación funcional sincronizada con el código real. En todos estos casos, la separación entre el metamodelo (estable) y los datos (variables) permite construir herramientas reutilizables a través de bancos y proyectos.

10. Conclusiones

Este trabajo planteó y sustentó la hipótesis de que las reglas de negocio bancarias, pese a su diversidad funcional aparente, comparten una estructura conceptual común caracterizable por un núcleo finito de siete dimensiones. La derivación del núcleo común a partir de la convergencia entre los estándares OMG SBVR y DMN, y el Business Rules Manifesto de Ronald Ross, dota a la propuesta de fundamento formal independiente del dominio bancario particular.

Sobre la base del núcleo común, se formuló un metamodelo semántico que materializa las siete dimensiones como tipos de nodos, aristas y propiedades en un grafo de conocimiento. El metamodelo incorpora explícitamente las cuatro capas características del framework Altamira (código, paramétrica, repositorio de componentes, orquestación batch), sin perder la base de teoría de compiladores heredada del Code Property Graph. El resultado es un metamodelo simultáneamente formal y bancariamente operativo.

La extracción automatizada de reglas sobre el grafo se realiza en dos tiempos: una consulta única identifica candidatos mediante un patrón estructural, y siete consultas adicionales mapean cada candidato a las dimensiones del núcleo común, produciendo un paquete contextual JSON estable y predecible. Las consultas son independientes del banco particular y dependen únicamente del metamodelo: cualquier sistema COBOL ingestado bajo el metamodelo definido puede ser consultado con el mismo conjunto de consultas, sin modificación.

El ejemplo completo presentado, basado en la operativa real de transferencia bancaria entre cuentas propias, verificó las tres hipótesis sobre un caso concreto: la estructura común se manifestó en las tres reglas embebidas, el metamodelo soportó uniformemente la representación de las cuatro capas, y la extracción en dos tiempos produjo paquetes contextuales completos y trazables.

Como líneas de trabajo futuro se identifican tres prioridades: (a) la implementación productiva de un ingestor que pueble el grafo a partir de código real, posiblemente apoyándose en herramientas como `cobol-rekt` [13]; (b) la extensión del detector para reconocer reglas residentes en datos paramétricos sin manifestación condicional en código; y (c) la evaluación cuantitativa del enfoque sobre un repositorio bancario completo, midiendo tasas de precisión y recall del detector frente a un conjunto de reglas anotadas manualmente por expertos funcionales.

La principal contribución del trabajo radica en proporcionar un puente operativo entre la teoría formal de reglas de negocio y la práctica industrial de modernización de sistemas heredados, mostrando que estructura común y diversidad operacional no son contradictorias

sino complementarias cuando se las articula adecuadamente a través de un metamodelo semántico y un lenguaje de consulta declarativo.

Referencias

- [1] Object Management Group. *Semantics of Business Vocabulary and Business Rules (SBVR), Version 1.5*. OMG Document Number formal/19-10-02, 2019.
- [2] Object Management Group. *Decision Model and Notation (DMN), Version 1.4*. OMG Document Number formal/22-10-01, 2022.
- [3] Ross, R. G. *The Business Rules Manifesto*. Business Rules Group, 2003. Disponible en <https://www.businessrulesgroup.org/brmanifesto.htm>.
- [4] Ross, R. G. *Principles of the Business Rule Approach*. Addison-Wesley Professional, 2003.
- [5] Yamaguchi, F., Golde, N., Arp, D., & Rieck, K. Modeling and Discovering Vulnerabilities with Code Property Graphs. En *Proceedings of the 2014 IEEE Symposium on Security and Privacy*, pp. 590-604, 2014.
- [6] Sneed, H. M. Object-Oriented COBOL Recycling. En *Proceedings of the Third Working Conference on Reverse Engineering (WCRE 1996)*, pp. 169-178, 1996.
- [7] Basel Committee on Banking Supervision. *Principles for Effective Risk Data Aggregation and Risk Reporting (BCBS 239)*. Bank for International Settlements, 2013.
- [8] Hay, D., & Healy, K. A. *Defining Business Rules: What Are They Really?* Business Rules Group, Final Report, Revision 1.3, 2000.
- [9] Cosentino, V., Cabot, J., Albert, P., Bauquel, P., & Perronnet, J. A Model-Based Approach to COBOL Code Analysis. En *Conference on Advanced Information Systems Engineering (CAiSE 2013)*, 2013.
- [10] Date, C. J. *What Not How: The Business Rules Approach to Application Development*. Addison-Wesley, 2000.
- [11] Pérez-Castillo, R., García-Rodríguez de Guzmán, I., & Piattini, M. Knowledge Discovery Metamodel ISO/IEC 19506: A Standard to Modernize Legacy Systems. *Computer Standards & Interfaces*, 33(6), 519-532, 2011.
- [12] Robinson, I., Webber, J., & Eifrem, E. *Graph Databases: New Opportunities for Connected Data*, 2nd Edition. O'Reilly Media, 2015.
- [13] Sen-Gupta, A. *cobol-rekt: An evolving toolkit of capabilities helpful for reverse engineering legacy Cobol code*. Repositorio GitHub, 2024. <https://github.com/avishek-sen-gupta/cobol-rekt>.